

Using Machine Learning to Assist Fund Selection

Introduction

I built a machine learning model, that ranks Indian active-equity funds within their category to help retail investors in choosing an equity fund easier. There are hundreds of active equity funds and their numbers are growing each year; this makes it nearly impossible for retail investors to compare all of them and choose one which would suit them best. Faced with hundreds of alternatives, investors often rely on simple decision heuristics rather than conducting comprehensive comparisons. One of the most common heuristics is selecting funds based primarily on their recent historical returns, despite extensive evidence that past performance alone is an unreliable predictor of future outperformance.

To help investors in making better choices, I set out to find patterns and correlations in the data that could help us rank the current active funds in the market by their future 3 year returns as well as put them in categories/baskets based on their risk profiles.

After collecting all the data, training the model and validating all my findings thoroughly, I was able to rank Indian equity funds within their own category with a min IC of 0.09 and max IC of 0.18 (above 0.1 is said to be good). To test this list, I chose 5 main categories – Small Cap, Mid Cap, Large Cap, Flexi Cap and ELSS and back tested on 7 years (from 2016 to 2022) by investing in the top 2 funds per category and rebalancing every year. This was compared against a benchmark – investing the same amount in every fund of those 5 categories. The result was a positive alpha in all 7 years with an average alpha of **2.4%/year**. This showed that the model had actual skill when ranking.

This does not mean, that it can beat the market, if the market goes down so will our returns, but it does show that compared to the benchmark, it had real skill in choosing which funds would perform better over the next 3 years.

Collecting The Data

All the data is cleanly stored in a Postgres Database:

- Millions of rows of NAV, AUM and TER data from AMFI. This is the price history, size and cost of each fund, which is where most of the features come from.
- Over 2.5 million portfolio holdings for both old and currently active funds, which is a record of what each fund actually owns.
- Every manager and their tenure for the active funds, with their start and end dates, so I could measure how experienced the managers are and whether it is a team or a solo manager.

Having this much historical data across hundreds of funds is what let the model actually find real patterns, instead of just memorising a few examples.

One thing I was careful about is that all the features are point-in-time. For each year-end snapshot, the inputs only use data from the previous 3 years (what we would have actually

known at that time), and the target looks at the next 3 years. For AUM and TER I always used the latest value on or before the snapshot date, never a future one. This is important because otherwise the model would accidentally get to "see the future," which would make the back-test results fake.

A few data limitations, and how I solved them

1. TER data was unreliable before 2018: Before 2018 there was no reliable TER, so those funds were given a neutral placeholder value. Because it's the same for every fund, the model simply can't split on it and just ignores it for those years, leaning on the other 8 features instead. It never uses a future value, so there's no look-ahead, and the feature-jackknife confirmed the model doesn't depend on TER anyway. Real TER from 2018 onward is where the cost feature actually does its work.

2. A single corrupted NAV was poisoning the model: One fund (HSBC Midcap) had a broken NAV series that showed up as a +4,367% return over 3 years, which is obviously wrong (the real historical max for an Indian equity fund is around 155%). The problem was that each fund's target is measured against its category average, so this one bad number pulled up the whole Mid-Cap 2021 benchmark and quietly ruined the target for all 19 mid-cap funds in that group. I fixed it using:

- A sanity filter: any 3-year return outside the range $[-95\%, +500\%]$ is treated as bad data and removed before it can affect the benchmark or the training set.
- A trimmed benchmark: the category average is calculated after dropping the top and bottom 5% of funds, so even if some other glitch slips through, one bad value can't take over the benchmark.

After these fixes, the worst distortion in any category went from 1.85 down to 0.16, so the training target became much cleaner.

The Model

I used a **LightGBM** (Light gradient boosting machine) – a distributed machine learning framework by Microsoft. It's in the same family like XGBoost in the sense it combines the predictions of many weak trees to create 1 strong model. The reason I used this is because it is said to be very efficient for structured, tabular data like financial and pricing models.

How I used It:

I gave the model a table, where each row is one fund at one point in time described by 9 numbers(input). I also tell the model what happened next -> did the model beat its own category over the following 3 years (target).

The model studies thousands of these and learns which combinations of the 9 numbers tend to perform the best and thus gives a score for **“how likely this fund is to beat its category”** then we simply rank by that score.

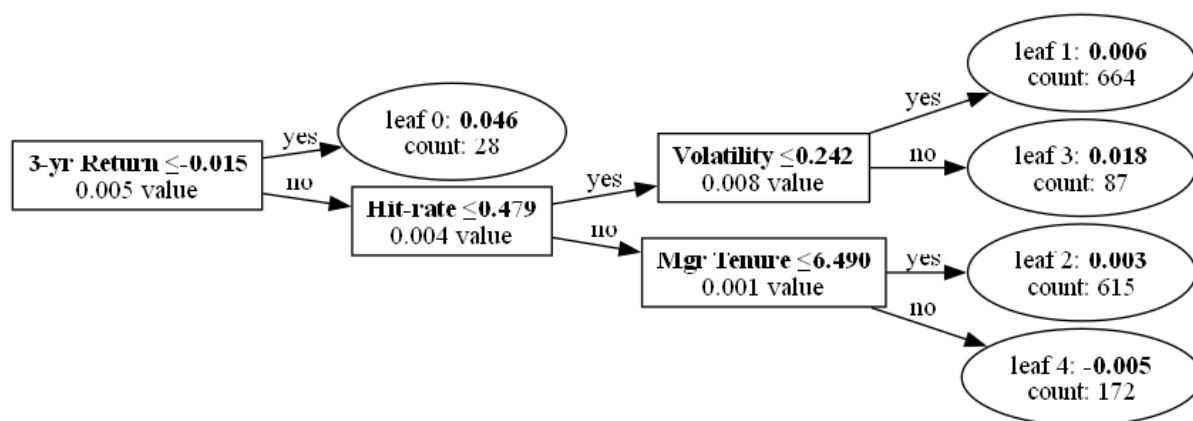
What we feed the model: one row = one fund at one point in time (9 input features -> 1 target)

Fund	Category	3-yr Return	Volatility	Hit-rate	Max Drawdown	AUM %ile	TER (cost)	Mgr Tenure	Beat category? (next 3y)
Invesco India Contra	Contra	57.6%	15.9%	58.3%	-13.7%	93.1%	0.98	0.0	-8.2%
Bandhan Elss Tax Saver	ELSS	44.3%	16.2%	58.3%	-16.0%	93.2%	1.23	2.2	16.3%
Bank Of India Elss Tax S	ELSS	35.7%	18.0%	55.6%	-20.6%	78.7%	1.23	0.0	49.2%
Invesco India Elss Tax S	ELSS	45.6%	15.3%	44.4%	-14.2%	88.9%	1.23	0.76	8.8%
Baroda Mid Cap Plan B	Mid Cap	20.1%	19.2%	50.0%	-19.7%	39.9%	1.23	0.0	20.0%
Navi Large Cap Equity	Large Cap	34.8%	14.5%	38.9%	-12.2%	75.2%	1.23	0.0	3.3%
Sundaram Select Focus	Focused	42.2%	13.5%	41.7%	-11.4%	88.5%	1.75	0.0	-9.3%

Blue = a few of the 9 INPUT features (what we know today) Gold = the TARGET (what happened next: did it beat its category over 3 years)

This way during training the algorithm notices patterns like whenever max drawdown is worse than 20%, future alpha tends to be negative.

So, it creates a split here -> [drawdown < -20%] and after training, the target column is removed and the model learns rules like if drawdown < -20 %, go left, if manager tenure > 5, go right.



The 9 features:

These features were derived from the 5 pillars of a fund's quality

- 1) **performance**
- 2) **risk**
- 3) **size**
- 4) **cost**
- 5) **people**

From performance we measure **how much** and **how consistent** – trailing 3-year returns and the historical hit-rate (the percentage of months this fund beat the average fund in its category). We measured risk using **historical volatility** and **max drawdown** (biggest loss from previous peak). The cost to an investor is the **TER** (Total expense ratio per year). The size of the fund is measured by **AUM Percentile**, this allows the model to capture **capacity constraint**, smaller funds can exploit small/midcap inefficiencies that large funds can't. The People are measured by **max-tenure** (how long the longest serving manager worked here) and the **number of managers**.

5 pillars of fund quality -> 9 machine-readable features



I tried also adding more features like fund flows but the model becomes slightly worse (-0.15%) probably since AUM data already contains that information.

Feature Importance:

There are 2 types:

- 1) **Split Importance** – how often the feature was used
- 2) **Gain Importance** – how much does this split reduce error

This helps us know which features actually reduced prediction error.

Feature importance, two ways: value of cuts (gain) vs how often used (split)

Feature	Pillar	Gain % (value of cuts)	# Splits	Split % (how often used)
3-yr Return	Performance	22.0%	25	10.0%
AUM %ile	Size	18.5%	57	22.9%
Volatility	Risk	13.2%	39	15.7%
TER (cost)	Cost	11.1%	30	12.0%
# Managers	People	10.4%	14	5.6%
Max Drawdown	Risk	10.1%	37	14.9%
Hit-rate	Performance	8.4%	21	8.4%
Mgr Tenure	People	6.3%	26	10.4%
Is-team	People	0.0%	0	0.0%

Gold = top by GAIN (3-yr Return: few cuts, each decisive) Blue = top by SPLIT (AUM %ile: the model's workhorse) Is-team: unused on both

Note: I could get rid of the is-team feature since it was unused.

The target:

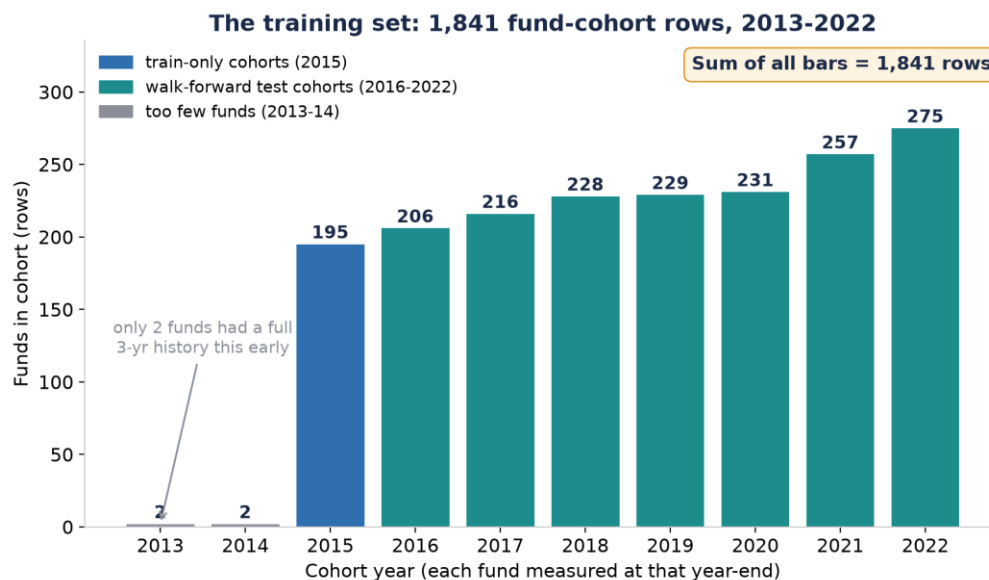
The target for prediction is **forward-alpha**, the fund's next 3-year return minus its **own categories'** average next 3-year return.

Eg: fwd_alpha = 0.05, it beat its category by 5% over 3 years.

The categories average future returns are calculated using a 5% trimmed mean (remove lowest and highest 5% and average what's left), this helps us remove strange values and prevent distorting the benchmark.

Training:

I picked only the **direct plan, growth option, equity asset class funds** to rank who also must have at least 30 months of historical features.



Thus, the model was trained on a wide span of market regimes.

The Model Configuration:

I have deliberately kept it small to guard against overfitting on a modest dataset (~1,841 rows):

- **max_depth = 3** - each tree asks at most three nested yes/no questions before reaching a decision. Shallow trees can only capture simple feature interactions, which stops any single tree from carving the data into tiny, over-fitted slices.
- **n_estimators = 50** - the model is built from 50 such trees in sequence (boosting): each new tree focuses on correcting the errors left by the ones before it, so the ensemble is far stronger than any individual weak tree.
- **Regression, not classification** - the model predicts a *continuous* score (expected 3-year category-relative alpha), which we then rank, rather than a simple beat/miss label. This preserves the *magnitude* of the signal, not just its direction.
- **random_state = 42** - fixed seed so every result is exactly reproducible.

Ranking:

The model outputs a single continuous number per fund which is its predicted 3-year category relative alpha. We convert that number into a within category percentile (**Quality, 0 -100**). A fund is ranked only within its own category.

Also with each fund, a **Risk Score** is calculated (based on volatility and max drawdown) to help us place each fund in a Conservative / Balanced / Aggressive band with a suggested holding period.

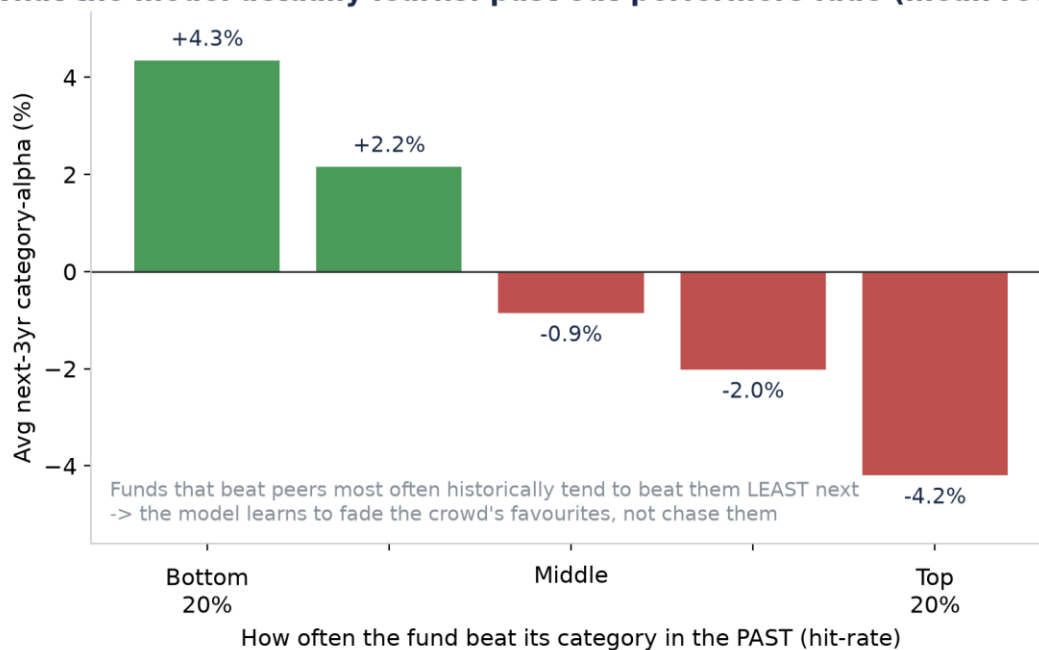
Explainability:

I used SHAP (**SHapley Additive exPlanations**), this helps us explain why and how much each feature pushed this specific fund's score up or down. The list thus shows the 2 highest impact features and translated them, into readable reasons for retail investors.

Note:

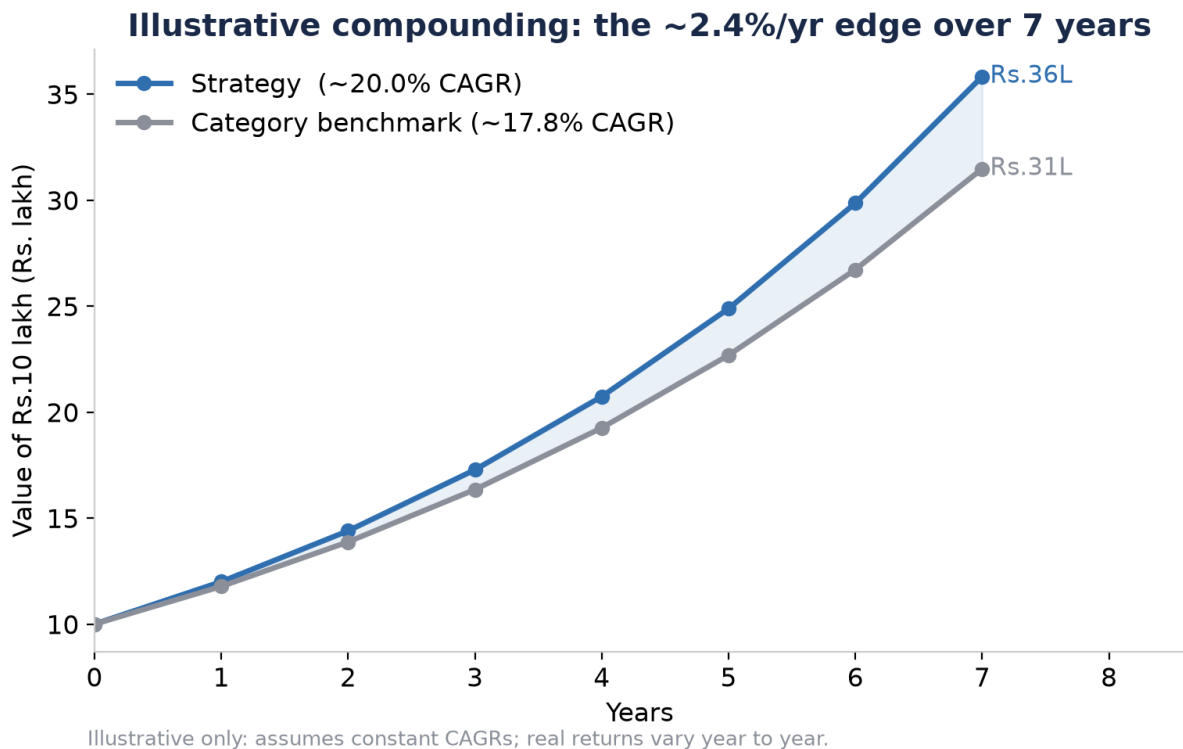
One pattern that surprised me is that it isn't just "high past returns = good." If anything the model learns the opposite in some cases: funds that beat their category the most in the past tend to beat it the least over the next 3 years (mean reversion). This is exactly why just picking last year's top performer doesn't work, and it's part of what the model is picking up on.

What the model actually learns: past out-performers fade (mean reversion)



Results

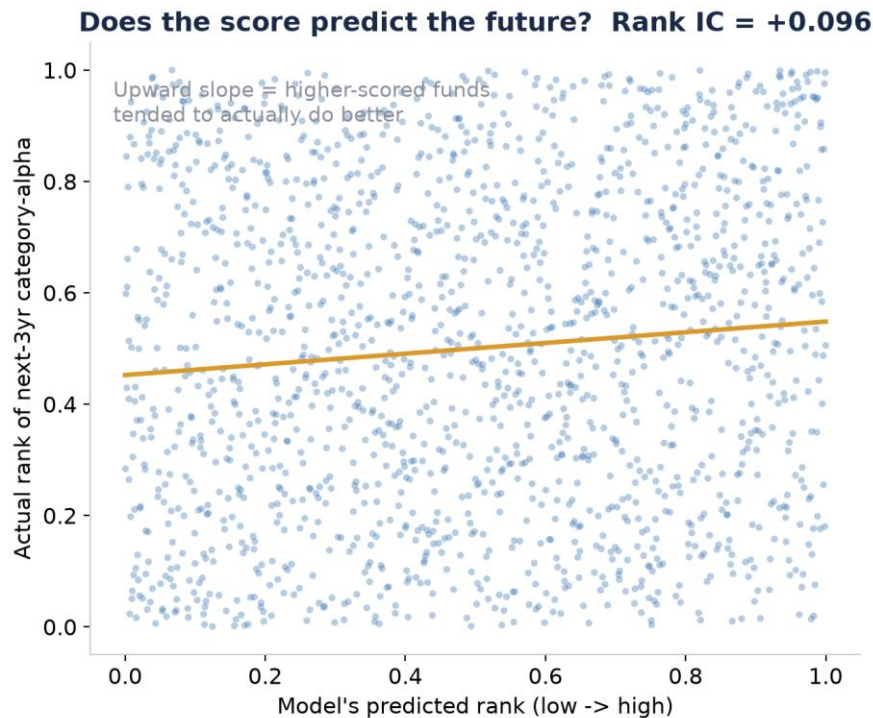
The final output is two things: a **retail fund list** (every fund rated and tiered within its own category, which is live on the website) and the **quant strategy** (the top-2-per-category portfolio I used to test the ranking). Over the back-test the strategy compounded at roughly **20% a year** against the benchmark's **~17.8%**, so about **2.4% a year on top** of a like-for-like benchmark.



illustrative growth of ₹10 lakh at 20% vs 17.8%, to show what the 2.4% edge is worth when it compounds.

The ranking actually works:

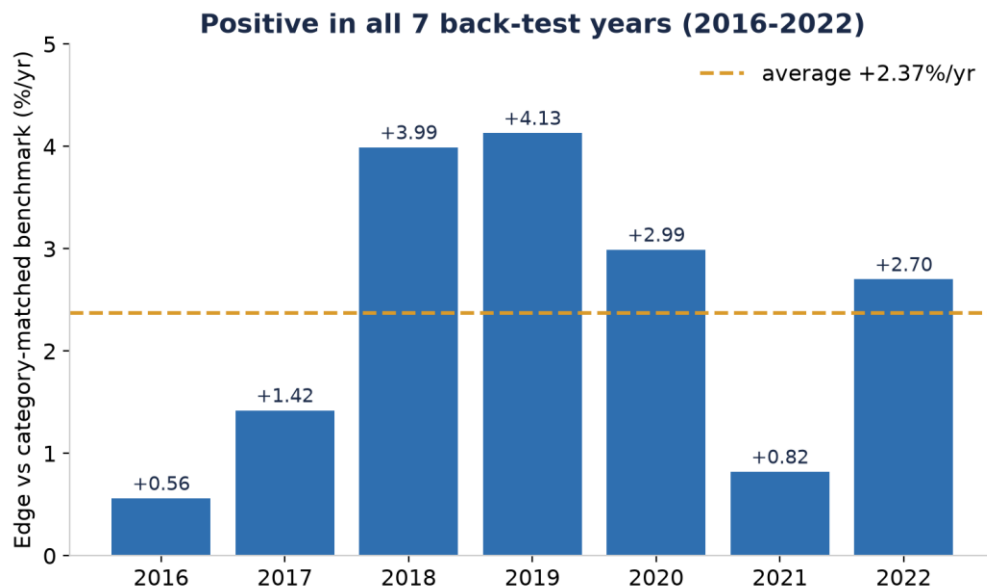
The first thing I checked was whether the score even predicts the future. Across all the test years, the rank IC (basically the correlation between the model's predicted ranking and how the funds actually did) came out to **+0.096 pooled, and up to +0.18 within a category**. Anything above 0.1 is considered good in this field. It's a weak signal per single fund (picking one fund is still close to a coin flip), but it's real and consistent, and it adds up across a whole portfolio.



predicted rank vs actual rank, showing the positive (if noisy) relationship, IC = +0.096.

Positive in all 7 years

When I actually ran the strategy year by year (buy the top-2 per category, rebalance every year, 2016 to 2022), it beat the category benchmark in **every single one of the 7 years**, with an average edge of **+2.37%/year**. The per-year edges were +0.56, +1.42, +3.99, +4.13, +2.99, +0.82 and +2.70%. So, it's not one lucky year carrying everything, it's consistent.



the 7 per-cohort alpha bars, all positive, with the +2.37% average line.

I also checked this properly with statistics, since the 3-year windows overlap and can make things look more significant than they are. After correcting for that (Newey-West), the edge is still significant (Newey West $t = 3.30$, $p \approx 0.001$), and a bootstrap gives a 95% confidence range of $[+1.37\%, +3.36\%]$, so even the pessimistic end is still positive.

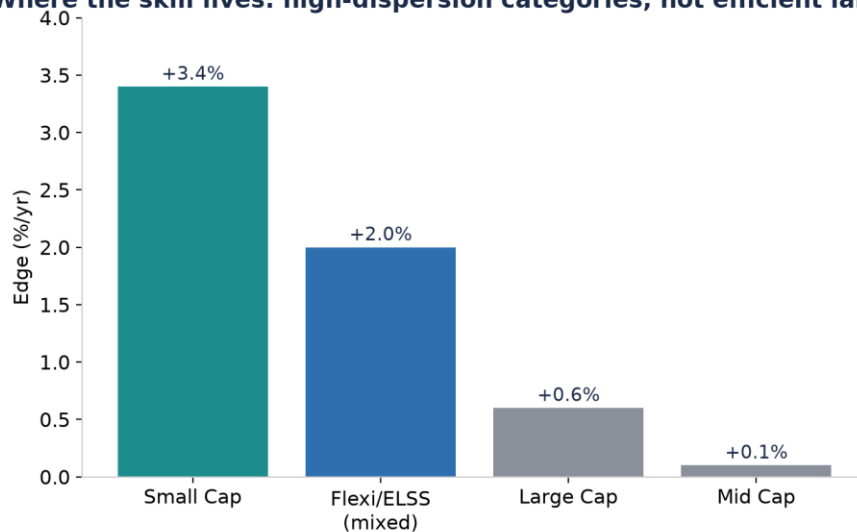
How much of this is real skill?

The +2.4% edge isn't all pure skill. Some of it comes from the fact that the model naturally leans a bit towards smaller funds, and smaller funds did well over this period (a "size" effect). To separate the two, I ran the returns through a factor model (market, size, momentum) and looked at what was left over after accounting for those tilts. What's left, the part that can't be explained by any tilt, is **+1.7%/year ($t = 4.26$)**, this is the genuine stock/fund-selection skill. So roughly **70% of the edge is real skill and about 30% is a small-fund tilt**, and momentum was basically zero (the model isn't just chasing recent winners).

Where the skill lives:

One honest point: the skill is not spread evenly. It's biggest in **small-cap (+3.4%/yr)**, where there's a lot of difference between funds and more room to pick well, and close to zero in **large-cap (+0.6%/yr)**, where most funds basically hug the index so there's little to choose between them.

Where the skill lives: high-dispersion categories, not efficient large-cap



edge by category, showing it's concentrated in small/mid, not large.

The Validations and Verifications

I spent most of the project actively trying to prove it was fake.

Negative control:

This was the most important test. The idea is simple: take the exact same strategy, but scramble the answers before training, so the model is learning from pure noise. If a strategy still "makes money" even when trained on random labels, then its edge was never real, it was just coming from the machinery (like a size tilt), not from any actual signal.

I ran this scrambled version 40 times with different random seeds to get a proper distribution, and then checked where the real model lands against it. The real model beat the scrambled versions almost every time ($p = 0.000$), so its edge genuinely comes from learning a real pattern, not from the setup

Red team:

Next, I ran a whole battery of checks, 6 in total, and it survived every one:

1. **Portfolio size** - I varied how many funds I hold (not just top-2). The edge holds across sizes, so it's not an artefact of one specific number.
2. **Random seed** - changing the model's random seed doesn't change the result, so it's not luck baked into one particular training run.
3. **Leave-one-year-out** - I dropped each test year one at a time. Even the worst case (dropping 2019) still gives +2.19%/yr, so no single year is carrying the whole result.
4. **Feature jackknife** - I removed each of the 9 features one at a time and retrained. The edge stays positive every time, worst case +1.38%/yr (dropping AUM). So it doesn't secretly depend on just one feature.
5. **"Is it just momentum?"** - I checked whether the model is really just buying whatever went up recently. It isn't: the picks aren't especially high-momentum, and the edge is basically unchanged (+2.41%) even after stripping momentum out.
6. **Random-portfolio placebo** - I built 2,000 random portfolios from the same funds and compared. The real strategy beats them with $p < 0.001$, so the result is not something you'd get by picking funds at random.

Is the benchmark fair?

I also tested the edge against harder benchmarks:

- **Cap-weighted benchmark** - instead of weighting every fund equally, I weighted the benchmark by fund size (AUM), which is a tougher comparison. The strategy still beats it, by +3.45%/yr, 7/7 years. So, the edge isn't an artefact of how I weighted the benchmark.
- **Factor-adjusted** - I ran the returns through a factor model (market, size, momentum) to strip out any "tilts." As I mentioned in Results, +1.72%/yr of genuine skill remains after all of that ($t = 4.26$). About 30% of the raw edge was a small-fund tilt, momentum was zero, and the rest is real selection skill.

Statistical significance

Because the 3-year return windows overlap, normal statistics would overstate how significant this is. So, I used the proper correction (Newey-West), and the edge is still significant ($t = 3.30$, $p \approx 0.001$). A bootstrap (resampling the years thousands of times) gives a 95% range of $[+1.37\%, +3.36\%]$, and a simple sign test (positive in 7 out of 7 years) gives $p = 0.008$. All three point the same way.

Limits:

- When I rebuild the benchmark to include dead and merged funds (to rule out survivorship bias), the edge drops only slightly, from the red team baseline of $+2.47\%$ to $+2.22\%$, still positive in all 7 years.
- But if I force every fund to have at least 5 years of history, the edge weakens to $+2.10\%$ and the negative control becomes borderline ($p = 0.050$). So part of the edge does rely on younger funds.

The Strategy

I didn't set out to build a strategy at all. I just wanted to know was whether the ranking skill was real. The way I tested that was to back-test it, buy the top-ranked funds each year and see if they beat the benchmark, and that back-test is basically a strategy in itself.

The way it works:

Top-2 per category. Each year, in each of the 5 core categories (Large Cap, Mid Cap, Small Cap, Flexi Cap and ELSS), I take the model's ranking and buy the top-2 funds. That's 10 funds, held equally.

Equal weight across all 5 categories. This is the important bit. I don't try to guess which category will do best next year, because honestly nobody knows that, and every time I let the model bet between categories it just re-introduced the small-fund tilt that failed the negative control. So, I put equal money in all 5 categories. This keeps the strategy focused on the one thing the model is actually good at, picking better funds *within* a category, and not on category-timing, which it isn't good at. This is also what makes the result "pure" selection skill rather than a style bet.

Rebalance once a year with a buffer. Every year I re-rank and update the holdings. But to avoid churning funds in and out (which costs tax and effort), I use a buffer rule: I only sell a fund when it drops out of the category's **top-4**, not the moment it falls below top-2. This cuts the turnover a lot (from 46% down to 33%) while still keeping almost all of the edge ($+2.98\%$ of a $+3.20\%$ edge).

Tax optimisation:

In India, if you sell an equity fund within a year the gains are taxed more heavily (short-term); hold longer than a year and it's taxed at the lower long-term rate. Because my strategy rebalances yearly, holding period really matters. Here's what I found:

Hold period	Gross edge	Net edge after tax	Turnover
1 year	+3.70%	+2.60%	~70%
2 years	+3.28%	+2.69% (best)	~44%
3 years	+2.47%	+1.90%	~29%

The tax drag is around 1.5–1.7%/year, but the key point is that it **does not erase the edge as long as you hold for more than 12 months**. The sweet spot is around a 2-year effective hold (which the buffer rule naturally pushes you towards), where the net edge is highest at +2.69%. Holding for under a year is where tax really hurts, so that's the one thing to avoid. (ELSS funds have a 3-year lock-in by law anyway, so they take care of themselves.)

So, the final strategy is: rank within each category, hold the top-2 per category equally across all 5 categories, rebalance yearly with a top-4 buffer, and hold each fund for at least a year (ideally 2) so tax doesn't eat the edge.

Findings and Experimentations

My research journey was a pretty arduous one. I had never built a project like this before, so I made a lot of mistakes and jumped to a lot of fast conclusions early on. But honestly, figuring out *why* those assumptions were wrong is where most of the actual learning happened. Here are the main ones.

"One feature to rule them all." My very first attempt wasn't even machine learning. I just ranked funds by their trailing 3-year return and bought the top ones, thinking past winners would keep winning. It underperformed a simple equal-weight benchmark. That was my first real lesson: chasing past returns doesn't work, because good funds mean-revert and a high return can be hiding a lot of risk-taking. This is actually what motivated the whole model, I needed something that could weigh returns *in context* (i.e. high momentum is only good if the drawdown is also controlled), which a single feature can never do.

Thinking I'd "beaten the market." My first full model looked amazing and I genuinely thought I'd beaten the market. Then I ran the negative control (train on scrambled labels) and it fell apart; the fake model did almost as well as my "real" one. That was a humbling moment, but it's the most important thing that happened in the whole project. It forced me to realise the edge wasn't market-beating alpha, it was a hidden small-fund tilt. Once I stopped trying to beat the market and instead measured skill *within* each category, I found the real (smaller, but genuine) edge.

After that, I spent a lot of time trying to make the edge *bigger*. Most of these didn't work, which was frustrating at the time, but it's actually useful information: it tells me the simple model is already close to the best I can do with this data. Here's what I tried:

- **Fund-flow features.** I added how much money was flowing in and out of each fund, thinking it might signal smart-money moves. The edge got slightly *worse* (−0.15%), probably because the AUM feature already captures most of that information. Dropped it.

- **Different targets (Sharpe / Sortino / Calmar).** Instead of ranking on future return-vs-category, I tried training the model to predict risk-adjusted measures like Sharpe and Sortino, thinking a "smarter" target would rank better. None of them beat the simple category-relative-alpha target. So, I kept the simple one.
- **LambdaMART (a learning-to-rank model).** This is a model built specifically for ranking problems rather than predicting a number and ranking afterwards, so I expected it to help. When I swept its settings, one lucky setting looked +0.81% better, but it only won in a minority of settings, so it wasn't a real, robust improvement, just noise. Rejected it.
- **Sharing information across categories (shrinkage).** I tried letting the categories "borrow" signal from each other (a hierarchical/Bayesian approach), thinking more data per category would help. It actually *hurt*, the skill turned out to be category-specific, so mixing categories together just watered it down.
- **De-tilted features.** This one is interesting because it half-worked. I re-scaled the features *within* each category-cohort, which pushed the ranking correlation (IC) up nicely, from ~0.09 to ~0.18. But when I put it into the actual top-2 strategy, the portfolio edge barely moved (the higher IC didn't translate into more money). So, for the strategy it was basically a dead end. But the same de-tilting turned out to be exactly what a plain "rate every fund" list needs, so this is what powers the separate retail list on the website. So, the one experiment that "failed" for the strategy is the one that gave me a second, genuinely useful product.

The Future

Use the portfolio holdings data. This is the big one. I already collected over 2.5 million portfolio holdings into the database, but the model doesn't actually use them yet, it currently judges a fund from the outside (its returns, size, cost, managers) without looking at *what it actually owns*. The holdings would let me build a whole new set of features: how concentrated a fund is, how much it overlaps with its peers, whether it's quietly drifting into sectors it isn't supposed to be in, and so on. This is the single most promising signal I haven't gotten to yet. The main blocker is linking the holdings cleanly to each fund, right now I can match roughly half of the recommended funds by their ISINs, and the rest need better name-matching. So, the next real step is closing that linkage gap and testing whether holdings-based features add to the edge.

Let the live paper-trade keep running. I've set up a live paper-trading account that tracks the strategy against a category benchmark using real daily NAVs. Everything so far has been a back-test on past data, so this is the first genuinely out-of-sample evidence, it's the fairest possible test because the model has never seen any of it. The longer it runs, the more I'll trust (or learn to doubt) the edge. Every new year also unlocks a fresh cohort to test on (2023's full 3-year result becomes available around 2027), so the evidence base keeps growing on its own.

Test it outside this one market and era. Everything here is Indian equity funds over a mostly bull-market period (2013–2025). Before I'd fully trust it, I'd want to see whether the

same approach holds in a different market or a proper downturn. That's the honest limit of a single-country, single-era result.

Add a passive benchmark. Right now, I compare funds against other funds. A useful addition would be comparing against index (passive) funds too, to answer the bigger question of whether active funds are even worth picking in the first place, for that I need clean index total-return history, which I haven't gotten to yet.